

1. Tutorial: definición de una visualización de horarios con el DSL

Este capítulo narra, paso a paso, el proceso de construir un programa completo en el DSL a partir de la descripción de un problema de horario.

La filosofía del DSL es **separar la descripción del problema de su generación**: el programador declara qué información mostrar (tablas, fórmulas, coloreado condicional) y el generador produce el programa que materializa esa visualización en el formato de salida deseado.

El ejemplo será un **horario de defensas de tesis**: dado un conjunto de profesores, estudiantes y locales, se necesita una aplicación que liste las defensas programadas, calcule automáticamente cuántas defensas acumula cada profesor y resalte en rojo a los profesores con conflicto de horario.

La generación a un libro Excel es solo una de las opciones que ofrece el DSL; el mismo programa podría producir, sin cambios, una hoja LibreOffice, un informe PDF o una tabla HTML.

Estructura del capítulo. En la Sección 1 se enumeran las herramientas necesarias. La Sección 2 presenta los conceptos fundamentales del DSL. En la Sección 3 se analiza el problema y se deciden las tablas. Las secciones 4 y 5 muestran cómo definir cada tabla con el DSL. La Sección 6 explica el formato de los datos de entrada. La Sección 7 ensambla el programa completo y la Sección 8 detalla cómo generar y ejecutar la aplicación.

1.1 Requisitos

Antes de comenzar conviene tener claro qué programas hay que instalar y cómo se organizan los archivos del proyecto.

1.1.1 Software necesario

Para utilizar el generador se necesitan dos programas:

Steel Bank Common Lisp (SBCL) Intérprete de Common Lisp. Disponible en <https://www.sbcl.org/>. Versión mínima recomendada: 2.0.

Python 3.10 o superior Para ejecutar el código generado. Además de Python estándar, se requiere la biblioteca **openpyxl**:

```
pip install openpyxl
```

1.1.2 Estructura del proyecto

El código fuente del generador se organiza en los siguientes archivos:

```
generador-excel/  
+-- dsl-directo.lisp          <- DSL y macros (no modificar)  
+-- ast-def.lisp             <- definicion de nodos AST (no  
    modificar)  
+-- generate-code-direct.lisp <- backend de generacion (no  
    modificar)  
+-- code-generation-utils.lisp  
+-- hoja_con_formulas.py      <- backend Python (no modificar)  
\-- mi-horario/  
    +-- data-tutorial.lisp    <- datos concretos (el usuario  
        crea este)  
    \-- tutorial-horario.lisp <- programa DSL (el usuario crea  
        este)
```

Los archivos de la raíz son la **biblioteca del generador** y no deben modificarse. El usuario únicamente crea su propio programa DSL y su archivo de datos.

Con el entorno listo, pasemos ahora a los conceptos que dan forma al DSL.

1.2 Conceptos fundamentales

1.2.1 El árbol de sintaxis abstracta (AST)

Cuando el usuario escribe definiciones en Common Lisp usando las macros del DSL, estas construyen automáticamente objetos en memoria que forman un **árbol de sintaxis abstracta (AST)**. Cada nodo del árbol representa un concepto del problema: una tabla, una expresión de celda, un cuantificador existencial, una regla de coloreado.

El usuario trabaja únicamente con las macros de alto nivel descritas en este documento; nunca necesita manipular el AST a mano. Esto es relevante porque libera al programador de tener que conocer la estructura interna del árbol: puede concentrarse en declarar qué quiere modelar y dejar que las macros construyan la representación intermedia automáticamente.

El AST es el puente entre el programa del usuario y el generador: una vez construido, puede alimentar cualquier backend. Esto nos lleva a una propiedad fundamental del DSL.

1.2.2 El DSL es independiente del generador de código

El AST no contiene ninguna referencia a Excel, a Python ni a ningún otro formato de salida concreto. Es una representación **neutral del problema**.

Un *generador de código* es cualquier programa que tome el AST como entrada, *recorra* sus nodos (es decir, los visite uno por uno en orden) y, para cada nodo, *interprete* su significado (decida qué instrucción o construcción emitir en el lenguaje de salida). El *destino* es el formato o plataforma para el que se genera código: Excel, LibreOffice, PDF,

HTML, entre otros. Así, cambiando solo el generador, el mismo AST produce resultados en destinos distintos.

En este trabajo el generador implementado toma el AST y produce un **script Python** que usa la biblioteca `openpyxl` para construir un archivo `.xlsx`. Pero el mismo DSL podría alimentar, sin cambio alguno en el programa del usuario, un generador diferente que produjera, por ejemplo, una hoja LibreOffice, un informe PDF o una tabla HTML.

El generador implementado en este trabajo

El backend concreto que se distribuye con el DSL genera un script Python que usa `openpyxl` para construir archivos Excel (`.xlsx`). Las fórmulas de celda, los datos y las reglas de formato condicional quedan incrustados en el archivo y funcionan en cualquier versión de Excel o LibreOffice Calc sin necesidad de tener instalado Python ni SBCL.

1.2.3 Etapas de la canalización

Una *canalización* (o *pipeline*) es una secuencia de etapas por las que pasa el programa desde que se escribe en el DSL hasta que se obtiene el resultado final. El proceso tiene tres etapas separadas:

1. **Construcción del AST.** Cada *forma* del DSL (cada instrucción como `def-table`, `exists`, `conditional-rendering`) se traduce a un nodo del árbol de sintaxis abstracta. Esta traducción la hacen las *macros* de Lisp: toman la forma escrita por el programador y la convierten en una instancia de una clase del AST con sus campos correspondientes. A esto se le llama “expandir” la forma.
2. **Generación de código.** El AST se pasa al generador, que recorre cada nodo y produce el programa de la aplicación en el formato de salida deseado. En el backend Excel que acompaña al DSL, ese programa es un script Python.
3. **Ejecución.** El programa generado se ejecuta y crea el archivo de salida con datos, fórmulas y reglas de formato condicional incrustadas.

Visto cómo funciona el DSL por dentro, pasemos ahora a modelar el problema concreto de las defensas de tesis.

1.2.4 Principio central: intención vs. implementación

El DSL declara *qué* debe ocurrir (“colorear los profesores con conflicto de horario”); el generador decide *cómo* implementarlo según el destino. Por ejemplo, para el backend Excel se genera una regla de formato condicional con la fórmula `SUMPRODUCT`. `SUMPRODUCT` es una función de Excel que multiplica matrices elemento a elemento y suma los resultados; aquí se usa para detectar fácilmente si dos defensas comparten profesor y horario. La regla *persiste* en el archivo `.xlsx` (queda incrustada, no se pierde al cerrar el libro) y *recalcula* automáticamente cada vez que se modifican los datos, sin necesidad de regenerar nada.

El programador del DSL se abstrae por completo de estos detalles: no escribe la fórmula `SUMPRODUCT`, no usa la sintaxis de Excel, no sabe si el backend genera fórmulas, colores fijos o algún otro mecanismo. Solo declara la condición con `exists` y el DSL se encarga

del resto.

Con estos fundamentos claros, apliquémoslos al problema concreto.

1.3 Del problema a las tablas

El primer paso es decidir cómo representar el dominio del problema en filas y columnas. Esta decisión es la más importante del modelado: de ella depende qué tablas hacen falta, qué columnas tiene cada una y cómo se relacionan entre sí.

1.3.1 *El problema del horario de defensas*

Se necesita planificar las defensas de tesis de una facultad. Cada defensa tiene un estudiante, un tribunal de cinco profesores (tutor, oponente, presidente, vocal y secretario), un día, una hora y un local. Además, se quiere saber cuántas defensas acumula cada profesor y detectar si algún profesor tiene dos defensas programadas a la vez.

A partir de esta descripción, el primer paso de diseño es decidir cómo organizar la información en tablas.

1.3.2 *Decidir cuántas tablas hacen falta*

Decisión de diseño

La pregunta clave es: *¿qué es una fila?*

Si una fila es **una defensa**, la tabla tiene tantas filas como defensas se deben planificar y sus columnas son: estudiante, los cinco roles del tribunal, día, hora y local.

Si una fila es **un profesor**, la tabla tiene tantas filas como profesores existen y necesita una columna calculada que cuente en cuántas defensas participa. En esta tabla habrá que agregar además columnas para el nombre y el grado académico de cada profesor.

La aplicación necesita *ambas* perspectivas. Por ese motivo se usan dos tablas en la misma hoja: la tabla de defensas y la tabla de profesores se referencian mutuamente (la segunda cuenta cuántas veces aparece cada profesor en la primera).

1.3.3 *Tabla de defensas*

La respuesta “una fila es una defensa” da lugar a la **tabla de defensas** (defensas-table):

Columna	Descripción
estudiante	Nombre del estudiante que defiende
tutor	Profesor tutor
oponente	Profesor oponente
presidente	Presidente del tribunal
vocal	Vocal del tribunal
secretario	Secretario del tribunal
dia	Día de la defensa
hora	Hora de inicio
local	Nombre del local

Por qué los cinco roles van juntos y en ese orden

Los roles `tutor`, `oponente`, `presidente`, `vocal` y `secretario` son columnas consecutivas. Esta disposición permite referenciar las cinco columnas como un solo rango — una secuencia continua de celdas— mediante la expresión (`trange defensas-table tutor secretario`). Así se cuenta cuántas veces aparece un profesor en cualquier rol con una sola llamada, sin tener que hacer cinco búsquedas independientes y sumarlas.

Ya tenemos la estructura conceptual. En la Sección 1.4 veremos cómo traducirla a código del DSL.

1.3.4 Tabla de profesores

La respuesta “una fila es un profesor” da lugar a la **tabla de profesores** (`profesores-table`):

Columna	Descripción
nombre	Nombre del profesor
grado	Grado académico
defensas	Número de defensas en que participa (calculada)

Esta tabla es más interesante que la anterior porque introduce tres características nuevas del DSL:

- **Columna calculada:** `defensas` no se introduce a mano sino que se obtiene mediante una fórmula que cuenta apariciones.
- **Referencia cruzada:** la fórmula busca en la tabla de defensas, no dentro de la propia tabla.
- **Formato condicional:** los profesores con conflicto de horario se colorean automáticamente.

Veremos cómo implementar cada una en la Sección 1.5.

1.4 Definir la tabla de defensas

Vamos a escribir el código DSL de las dos tablas. Todo este código se coloca en el archivo `tutorial-horario.lisp` que crea el usuario. Empecemos por la tabla de defensas, que

es la más sencilla.

La macro principal para definir una tabla es `def-table`:

Sintaxis: `def-table`

```
1 (def-table nombre-tabla
2   ((col1 "Cabecera1") (col2 "Cabecera2") ...))
3   :cell-height N
4   :cell-width N)
```

Cada par (columna "Cabecera") declara dos cosas:

- **columna** es el *nombre interno* (un símbolo del DSL) que se usa para referirse a esa columna en expresiones como `(col nombre)`, `(trange ... nombre)` o `(param nombre)`.
- "Cabecera" es el *texto del encabezado visual*, es decir, la etiqueta que se muestra en la fila de encabezados de la tabla en el archivo de salida. El `"` en el código indica que el encabezado se rellenará más adelante desde los datos de entrada.

Las opciones `:cell-height` y `:cell-width` controlan cuántas unidades físicas (filas o columnas de la cuadrícula de la aplicación) ocupa cada fila o columna lógica del modelo (el valor 1 es el caso habitual; valores mayores sirven para tablas con celdas que abarcan varias filas o columnas en la representación visual).

Apliquemos esta sintaxis a la tabla de defensas.

Listing 1.1: Definición de `defensas-table`.

```
1 (def-table defensas-table
2   ((estudiante "") (tutor "") (oponente "") (presidente ""))
3   (vocal "") (secretario "") (dia "") (hora "") (local ""))
4   :cell-height 1
5   :cell-width 1)
```

El valor `""` en cada par es un marcador que indica que esa columna no tiene un valor fijo: se rellenará con datos externos o con el resultado de una fórmula. El DSL ofrece también una alternativa más expresiva: la constante `*empty*` y el macro `empty` (que veremos en la Sección 1.5) representan el vacío semántico de forma explícita, y cada backend decide cómo traducirlo (en Excel/Python produce `""`, pero en otros formatos podría ser `null`, `nil` o simplemente omitir la celda).

1.5 Definir la tabla de profesores

La tabla de profesores introduce tres características nuevas: columnas calculadas, referencias cruzadas y formato condicional. Veamos cada una por separado, empezando por la columna calculada.

1.5.1 Columna calculada con `:computed`

La columna `defensas` de la tabla de profesores no se introduce a mano: queremos que se calcule automáticamente contando en cuántas defensas participa cada profesor. En lenguaje natural sería: “para cada profesor, cuenta cuántas filas de la tabla de defensas contienen su nombre en cualquiera de los cinco roles”. El DSL expresa esto con una columna *calculada*.

La opción `:computed` asocia columnas a **expresiones DSL**, que son las formas que proporciona el lenguaje para describir cálculos (`countif`, `trange`, `col`, `str`, entre otras). El generador traduce cada expresión al mecanismo apropiado según el destino (una fórmula de celda en Excel, una expresión en otros formatos). El valor de esa celda en los datos de entrada se ignora: la celda mostrará siempre el resultado de la expresión.

El código siguiente aplica esta opción a la tabla de profesores:

Listing 1.2: Columna defensas calculada con COUNTIF.

```
1 (def-table profesores-table
2   ((nombre "") (grado "") (defensas ""))
3   :cell-height 1
4   :cell-width 1
5   :computed
6   ((defensas (countif (trange defensas-table tutor secretario
7                        )
                        (col nombre)))))
```

`countif` es una expresión DSL que cuenta cuántas celdas de un rango cumplen un criterio. El primer argumento es el rango donde buscar (`trange`); el segundo es el valor que se compara (`(col nombre)`), que toma el nombre del profesor de la fila actual).

Para entender el rango sobre el que se aplica `countif`, es necesario explicar primero el mecanismo de referencias entre tablas.

Cómo funciona `trange`

Un *rango* es un bloque rectangular de celdas de una hoja de cálculo. `trange` (*table range*) crea un rango que abarca todas las filas de datos de *otra* tabla, desde una columna de inicio hasta una columna de fin. En concreto, `(trange defensas-table tutor secretario)` se refiere a todas las filas de `defensas-table` desde la columna `tutor` hasta la columna `secretario`. Como las cinco columnas de tribunal son consecutivas, una sola expresión alcanza para cubrirlas a todas (de no ser consecutivas, harían falta varias expresiones `countif` sumadas).

Se distingue de `range` en que `range` se usa dentro de la *misma* tabla (abrevia la tabla actual), mientras que `trange` nombra explícitamente la tabla de origen. Esto evita ambigüedades cuando dos tablas en la misma hoja comparten nombres de columna (como `nombre`, que existe en ambas tablas) y, sobre todo, deja claro que la referencia cruza los límites de la tabla.

Además de calcular valores automáticamente, el DSL permite definir reglas de formato condicional. Veamos cómo.

1.5.2 Formato condicional con `:render`

El **formato condicional** (o *conditional rendering*) es un mecanismo que cambia la apariencia de una celda (color de fondo, fuente, bordes, etc.) cuando se cumple una condición determinada. Permite que ciertos valores destaquen visualmente sin necesidad de revisar fila por fila. En el horario de defensas, queremos que los profesores con conflicto de horario aparezcan resaltados. En lenguaje natural: “si existe alguna defensa en la que este profesor participa y que comparte el mismo día y hora con otra defensa, colorea su nombre”.

La opción `:render` acepta una forma `conditional-rendering` que especifica bajo qué condición se colorean determinadas columnas. La condición se expresa con el cuantificador `exists`.

Listing 1.3: Formato condicional sobre profesores-table.

```
1 (def-table profesores-table
2   ((nombre "") (grado "") (defensas ""))
3   :cell-height 1
4   :cell-width 1
5   :computed
6     ((defensas (countif (trange defensas-table tutor secretario
7                           )
8                           (col nombre))))
9   :render
10    (conditional-rendering
11     :condition
12       (exists (r :rows-of defensas-table)
13         :self-in (tutor oponente presidente vocal secretario
14                   )
15         :matching (dia hora))
16     :target-columns (nombre)))
```

La variable `r` es un **alias** (un nombre corto y temporal) que representa cada fila de `defensas-table` a medida que `exists` las va examinando una por una. Es análogo a un alias de tabla en SQL: permite referirse a las columnas de la fila que se está comprobando en cada iteración, aunque en este ejemplo no se usa `r` explícitamente porque el DSL ya sabe qué tabla está en contexto. El cuantificador `exists` recorre todas las filas de `defensas-table` y se cumple si encuentra al menos una que satisfaga la condición.

Sintaxis: `exists :rows-of`

```
1 (exists (var :rows-of tabla-id)
2   :self-in (col1 col2 ...)
3   :matching (clave1 clave2 ...))
```

Parámetro	Significado
<code>:rows-of</code>	Tabla en la que se busca
<code>:self-in</code>	Lista de columnas de esa tabla donde debe aparecer el valor de la celda actual
<code>:matching</code>	Columnas cuyo slot debe tener más de una defensa programada (conflicto)

Veamos un ejemplo concreto. Supongamos que el profesor **Piad** aparece como tutor de la primera defensa y como presidente de la segunda. El cuantificador **exists** hace lo siguiente:

1. Toma la fila actual de **profesores-table** (“Piad”).
2. Recorre cada fila de **defensas-table** buscando una donde **Piad** aparezca en alguno de los roles listados en **:self-in** (tutor, oponente, presidente, vocal, secretario).
3. Para cada fila donde aparece, comprueba si *otra* fila distinta tiene el mismo par (**dia**, **hora**) (**:matching**), es decir, si hay dos defensas simultáneas en las que participa el mismo profesor.
4. Si existe al menos una fila así, la condición se cumple y el nombre **Piad** se colorea.

En este caso, **Piad** aparece en dos defensas (una como tutor el lunes a las 10:00 y otra como presidente el lunes a las 14:00). Como los horarios son distintos, no hay conflicto y **Piad** no se colorea. Si ambas defensas hubieran sido el mismo día y hora, la condición se habría activado.

El formateo condicional es una regla (no una instantánea)

El generador no aplica el color en el momento de generar: incrusta una regla en el archivo de salida que se reevalúa automáticamente cada vez que los datos cambian. Así, si se añade o modifica una defensa, los colores se actualizan sin necesidad de volver a ejecutar el DSL.

Con las tablas definidas, el siguiente paso es preparar los datos que alimentarán la aplicación.

1.6 Preparar los datos

Los datos se guardan en un archivo separado (**data-tutorial.lisp**) como parámetros globales de Lisp. Cada parámetro es una lista de listas: una sublista por fila.

1.6.1 Formato de los datos

Cada tabla espera **tres filas de prefijo** (llamadas así porque se colocan delante de los datos) seguidas de las filas de datos. Las filas de prefijo se escriben tal cual en el archivo **data-tutorial.lisp** (no generan fórmulas): la primera puede llevar un identificador opcional (para etiquetar el bloque de datos, aunque en este tutorial se deja vacío), la segunda está vacía y la tercera contiene los encabezados de columna.

Listing 1.4: Datos de defensas-table.

```

1 (defparameter *DATOS-DEFENSAS*
2   '(((" " " " " " " " " " " " " ")      ;; fila 1: prefijo
3     (" " " " " " " " " " " " " ")      ;; fila 2: separador
4     ("Estudiante" "Tutor" "Oponente"      ;; fila 3: encabezados
5      "Presidente" "Vocal" "Secretario" "Dia" "Hora" "Local")
6     ("Juan Diaz" "Piad" "Garcia" "Lopez" "Torres" "Ruiz"
7      "Lunes" "10:00" "Aula
8      1")
9     ("Maria Vega" "Garcia" "Lopez" "Piad" "Soto" "Torres"
10      "Lunes" "14:00" "Aula
11      2")
12     ("Luis Mora" "Lopez" "Torres" "Garcia" "Ruiz" "Soto"
13      "Martes" "10:00" "Aula
14      1"))))

```

Las celdas que corresponden a columnas calculadas se marcan con `*empty*` (o simplemente se dejan como `)` en el archivo de datos: el generador las sobrescribe con la fórmula correspondiente. La ventaja de usar `*empty*` es que hace explícita la intención: “esta celda no está vacía por accidente, sino porque su valor lo determinará una fórmula”.

Listing 1.5: Datos de profesores-table.

```

1 (defparameter *DATOS-PROFESORES*
2   '(((" " " " " ")
3     (" " " " " ")
4     ("Nombre" "Grado" "Defensas")
5     ("Piad" "Dr" " ")      ;; "Defensas" se calcula con
6     COUNTIF
7     ("Garcia" "Dr" " ")
8     ("Lopez" "Msc" " ")
9     ("Torres" "Msc" " ")
10    ("Ruiz" "Lic" " ")
11    ("Soto" "Msc" " ")))

```

Una vez que tenemos las tablas definidas y los datos listos, el siguiente paso es ensamblar el programa completo.

1.7 Construir la aplicación

Con las tablas declaradas y los datos preparados, se ensambla la aplicación con tres macros: `libro`, `hoja` y `tabla`.

Sintaxis: `libro / hoja / tabla`

```

1 (libro nombre-app
2   :filename "salida.xlsx"
3   :hojas (list
4     (hoja "Nombre hoja"
5       (tabla plantilla1 :data *DATOS-1*))

```

```

6      (tabla plantilla2 :data *DATOS-2*)
7      :fixed-expressions
8          (((nav ancla dir N) (expresion))
9              ...)))

```

La clave `:filename` indica el nombre del archivo de salida y es propia de los backends que exportan a un fichero (como el backend Excel que acompaña al DSL); un generador diferente para, por ejemplo, HTML o PDF, usaría otros mecanismos para producir la salida.

La macro `def-table` define una **plantilla** reutilizable (el equivalente a declarar una clase sin instanciarla): describe la estructura de la tabla (columnas, fórmulas, formato) pero no contiene datos propios. La macro `tabla` instancia esa plantilla con datos concretos. De esta forma, una misma plantilla puede usarse varias veces con distintos conjuntos de datos, igual que una clase da lugar a múltiples objetos.

Cuando una hoja recibe más de una `tabla`, el DSL las agrupa en una misma **región** (un conjunto de tablas que se presentan juntas). Cómo se coloca esa región en la interfaz generada es decisión de cada backend: el backend Excel de este trabajo las sitúa una al lado de otra separadas por un espacio en blanco; otro backend podría apilarlas verticalmente, mostrarlas en pestañas independientes, o distribuirlas según las dimensiones de cada tabla.

La clave `:fixed-expressions` coloca expresiones o etiquetas fuera de las tablas. La posición se especifica con `nav`, que indica una posición relativa a la última o primera fila de una tabla.

Listing 1.6: Aplicación completa con totales de pie de tabla.

```

1  (libro horario-tesis
2      :filename "Horario_Tesis.xlsx"
3      :hojas (list
4          (hoja "Defensas"
5              (tabla defensas-table :data *DATOS-DEFENSAS*)
6              (tabla profesores-table :data *DATOS-PROFESORES*)
7              :fixed-expressions
8                  (((nav (ultima-fila defensas-table estudiante) abajo 1)
9                      (str "Total defensas:"))
10                     ((nav (ultima-fila defensas-table estudiante) abajo 1
11                             derecha 1)
12                         (counta (trange defensas-table estudiante)))
13                     ((nav (ultima-fila profesores-table nombre) abajo 1)
14                         (str "Total profesores:"))
15                     ((nav (ultima-fila profesores-table nombre) abajo 1
16                             derecha 2)
17                         (sum-range (trange profesores-table defensas)))))))

```

La posición de cada expresión respecto a las tablas se especifica con la forma `nav`, que se describe a continuación.

Sintaxis: `nav / ultima-fila`

```
1 ;; Posicion relativa a la ultima fila de una columna:
2 (nav (ultima-fila tabla-id columna) abajo N)
3 (nav (ultima-fila tabla-id columna) abajo N derecha M)
4 (nav (primera-fila tabla-id columna) arriba N)
```

Por qué usar `nav` en lugar de coordenadas fijas

La posición de una expresión como “una fila debajo de la última defensa” depende de cuántas filas de datos haya. `nav` la calcula automáticamente en tiempo de generación, de modo que si se añaden más filas los totales se desplazan solos sin modificar el código escrito en el DSL.

Con la aplicación armada, solo falta generar y ejecutar el programa de construcción para obtener el archivo de salida.

1.8 Generar y ejecutar

Las dos últimas líneas del programa desencadenan la generación del archivo:

Listing 1.7: Compilacion y ejecucion desde el programa DSL.

```
1 (xl-generate horario-tesis "horario-tesis.py")
2 (xl-run-generated "horario-tesis.py")
```

`xl-generate` recorre el AST de la aplicación y produce el programa de construcción para el formato elegido. `xl-run-generated` ejecuta ese programa con `python3`; el resultado es el archivo `Horario_Tesis.xlsx` en disco.

El archivo `.xlsx` contiene tanto las fórmulas de celda como las reglas de formato condicional; estas últimas permanecen activas cuando el usuario abre y edita el archivo directamente en Excel.

1.8.1 Ejecución desde la terminal con el script `generar.sh`

Como alternativa a invocar SBCL directamente, el directorio incluye un script Bash `generar.sh` que automatiza el mismo flujo:

```
# Primera vez: dar permisos de ejecucion
chmod +x generar.sh

# Ejecutar el flujo completo
./generar.sh
```

El script realiza exactamente los mismos tres pasos que ocurren al cargar el programa DSL a mano:

1. Sitúa el directorio de trabajo en la carpeta del tutorial (para que los `load` relativos del programa DSL funcionen independientemente del directorio desde el que se llame

al script).

2. Invoca `sbcl --script tutorial-horario.lisp`, que expande las macros, construye el AST, genera `horario-tesis.py` y lo ejecuta con `python3`.
3. Verifica que `Horario_Tesis.xlsx` fue creado e imprime su tamaño.

¿Cuándo usar el script y cuándo cargar directamente?

El script es conveniente para ejecutar el tutorial de un solo paso desde cualquier directorio. Cargar el archivo directamente desde el REPL de SBCL (`(load "tutorial-horario.lisp")`) es preferible durante el desarrollo, porque permite inspeccionar el estado del AST en memoria entre las distintas formas del DSL.

1.9 El programa completo

A continuación se presenta el contenido íntegro de los dos archivos que conforman el programa.

1.9.1 *data-tutorial.lisp*

Listing 1.8: `data-tutorial.lisp` — datos de entrada.

```
1 (defparameter *DATOS-DEFENSAS*
2   '(((" " " " " " " " " " " " " ")
3     (" " " " " " " " " " " " ")
4     ("Estudiante" "Tutor" "Oponente" "Presidente" "Vocal"
5      "Secretario" "Dia" "Hora" "Local")
6     ("Juan Diaz" "Piad" "Garcia" "Lopez" "Torres" "Ruiz"
7      "Lunes" "10:00" "Aula 1")
8     ("Maria Vega" "Garcia" "Lopez" "Piad" "Soto" "Torres"
9      "Lunes" "14:00" "Aula 2")
10    ("Luis Mora" "Lopez" "Torres" "Garcia" "Ruiz" "Soto"
11     "Martes" "10:00" "Aula 1"))))
12
13 (defparameter *DATOS-PROFESORES*
14   '((" " " " ,*empty*)
15     (" " " " ,*empty*)
16     ("Nombre" "Grado" "Defensas")
17     ("Piad" "Dr" ,*empty*) ;; "Defensas" se calcula
18     con COUNTIF
19     ("Garcia" "Dr" ,*empty*)
20     ("Lopez" "Msc" ,*empty*)
21     ("Torres" "Msc" ,*empty*)
22     ("Ruiz" "Lic" ,*empty*)
23     ("Soto" "Msc" ,*empty*)))
```

1.9.2 *tutorial-horario.lisp*

Listing 1.9: tutorial-horario.lisp — programa DSL completo.

```

1 (load "dsl-directo.lisp")
2 (load "data-tutorial.lisp")
3
4 ;; TABLA 1: defensas
5 ;; Nueve columnas: cinco roles consecutivos (tutor...secretario
6   ),
7 ;; dia, hora, local. Sin coloreado propio.
8 (def-table defensas-table
9   ((estudiante "") (tutor "") (oponente "") (presidente "")
10    (vocal "") (secretario "") (dia "") (hora "") (local ""))
11   :cell-height 1
12   :cell-width 1)
13
14 ;; TABLA 2: profesores
15 ;; Columna "defensas" calculada con COUNTIF.
16 ;; Coloreado condicional: nombre en rojo si el profesor tiene
17   conflicto.
18 (def-table profesores-table
19   ((nombre "") (grado "") (defensas ""))
20   :cell-height 1
21   :cell-width 1
22   :computed
23   ((defensas (countif (trange defensas-table tutor secretario
24   )
25   (col nombre))))
26   :render
27   (conditional-rendering
28     :condition
29     (exists (r :rows-of defensas-table)
30       :self-in (tutor oponente presidente vocal secretario
31       )
32       :matching (dia hora))
33     :target-columns (nombre)))
34
35 ;; WORKBOOK
36 (libro horario-tesis
37   :filename "Horario_Tesis.xlsx"
38   :hojas (list
39     (hoja "Defensas"
40       (tabla defensas-table :data *DATOS-DEFENSAS*)
41       (tabla profesores-table :data *DATOS-PROFESORES*)
42       :fixed-expressions
43       (((nav (ultima-fila defensas-table estudiante) abajo 1)
44         (str "Total defensas:"))
45        ((nav (ultima-fila defensas-table estudiante) abajo 1
46         derecha 1)
47         (counta (trange defensas-table estudiante)))
48        ((nav (ultima-fila profesores-table nombre) abajo 1)
49         (str "Total profesores:"))

```

```

45      ((nav (ultima-fila profesores-table nombre) abajo 1
46           derecha 2)
47           (sum-range (trange profesores-table defensas))))))
48 (xl-generate horario-tesis "horario-tesis.py")
49 (xl-run-generated "horario-tesis.py")

```

1.10 Resultado esperado

El programa genera `Horario_Tesis.xlsx` con la siguiente estructura:

Rango	Contenido
Columnas A–I, filas 4–6	<code>defensas-table</code> : tres defensas
Columna J	Separador en blanco
Columnas K–M, filas 4–9	<code>profesores-table</code> : seis profesores
Columna M, filas 4–9	Fórmulas <code>COUNTIF</code> generadas
Fila 7 (cols. A–B)	Totales de defensas (<code>COUNTA</code>)
Fila 11 (cols. K, M)	Totales de profesores (<code>SUM</code>)
Rango K4:K9	<code>FormulaRule</code> : nombres en rojo si hay conflicto

Con los datos de ejemplo, las defensas de “Juan Díaz” y “Luis Mora” comparten el slot **Lunes 10:00** con roles solapados, por lo que los profesores Piad, García, López, Torres, Ruiz y Soto aparecen resaltados en rojo.

1.11 Traza del diseño

El cuadro siguiente resume las decisiones tomadas durante el modelado, relacionando cada requisito del problema con su representación en el DSL.

Requisito	Representación	Decisión de diseño
Una fila por defensa con tribunal completo	<code>defensas-table</code>	Granularidad natural; agrupa todas las columnas de tribunal
Los cinco roles en columnas consecutivas	<code>orden</code> en <code>def-table</code>	Permite <code>trange tutor secretario</code> para contar roles
Contar defensas por profesor	<code>columna</code> <code>computed</code>	<code>countif</code> sobre el rango de roles de <code>defensas-table</code>
Detectar profesores con conflicto	<code>:render</code> con <code>exists</code> <code>:rows-of</code>	El backend genera una regla de formateo condicional que persiste
Totales dinámicos al pie de tabla	<code>:fixed-expression</code> con <code>nav</code>	Las coordenadas se recalculan si cambia el número de filas

1.12 Referencia rápida de formas

Forma	Descripción
<code>(def-table nom cols opts...)</code>	Define una plantilla de tabla
<code>:computed ((col expr))</code>	Columna calculada (el backend la traduce al mecanismo adecuado)
<code>:render (conditional-rendering)</code>	Regla de formateo condicional
<code>(conditional-rendering :condition :target-columns)</code>	Asocia condición con columnas a formatear
<code>(exists (r :rows-of tabla) ...)</code>	Existencia en otra tabla (tabla cruzada)
<code>(exists (r :all-rows) ...)</code>	Existencia en la misma tabla
<code>(col nombre)</code>	Valor de la columna en la fila actual
<code>(trange tabla desde hasta)</code>	Rango en otra tabla
<code>(countif rango criterio)</code>	Cuenta celdas que cumplen un criterio
<code>(counta rango)</code>	Cuenta celdas no vacías
<code>(sum-range rango)</code>	Suma los valores de un rango
<code>(str "texto")</code>	Literal de texto
<code>(nav (ultima-fila t c) dir N)</code>	Posición relativa a última fila
<code>(nav (primera-fila t c) dir N)</code>	Posición relativa a primera fila
<code>(libro nom :filename :hojas)</code>	Define la aplicación completa
<code>(hoja nom tablas :fixed-expressions)</code>	Define una hoja de la aplicación
<code>(tabla plantilla :data datos)</code>	Instancia una plantilla con datos
<code>(empty)</code>	Vacío semántico: el backend decide cómo representarlo
<code>(xl-generate wb archivo)</code>	Genera el programa de construcción
<code>(xl-run-generated archivo)</code>	Ejecuta el programa y produce la salida

1.13 Formas del DSL no utilizadas en este tutorial

El tutorial cubre un subconjunto del DSL suficiente para construir el horario de defensas. Las formas que se describen a continuación están disponibles y resultan útiles en escenarios más complejos.

Vacío semántico: `empty` y `*empty*`

El DSL distingue entre “una celda que está vacía porque no tiene datos” y “una celda que se deja vacía porque su valor lo determinará una fórmula”. Para este segundo caso existen dos formas:

Forma	Para qué sirve
<code>(empty)</code>	Macro para usar dentro de expresiones DSL (<code>:computed</code> , <code>_if</code> , etc.). El backend decide la representación concreta.
<code>*empty*</code>	Constante para usar en datos y cabeceras: <code>(columna *empty*)</code> o <code>("PiadDr"*empty*)</code> . Hace explícito que la celda se rellenará con una fórmula.

Condicionales en celdas

Permiten que el valor de una columna dependa de una condición evaluada fila a fila.

Forma	Para qué sirve
<code>(_if cond entonces sino)</code>	IF de tres ramas. Muestra un valor sólo cuando se cumple la condición; en caso contrario muestra otro.
<code>(non-empty expr)</code>	Verdad si la expresión no está vacía (<code><></code>). Útil como condición de <code>_if</code> .
<code>(show-nothing)</code>	Produce la cadena vacía <code>.</code> Se usa como rama <i>sino</i> para dejar la celda en blanco.
<code>(equals a b) / (different a b)</code>	Igualdad y desigualdad entre dos expresiones.
<code>(_and a b) / (_or a b)</code>	Conjunción y disyunción lógica.

Ejemplo: mostrar el nombre de una sala sólo si la fila tiene un evento asignado, y dejarla en blanco en caso contrario.

```
1 (col "Sala" :computed
2   (_if (non-empty (col "Evento"))
3     (col "Sala")
4     (show-nothing)))
```

Navegación entre filas

Permiten que una columna acceda al valor de la misma celda en la fila anterior o siguiente, lo que es indispensable para calcular diferencias temporales entre registros consecutivos.

Forma	Para qué sirve
(previous-row expr)	Desplaza la referencia una fila hacia arriba.
(previous-of base)	Valor de base en la fila inmediatamente anterior.
(next-of base)	Valor de base en la fila siguiente.
(it-is-the-first-row)	Verdad en la primera fila de datos. Se combina con <code>_if</code> para el caso borde donde no existe fila anterior.
<code>primera-fila</code>	Ancla de posición a la primera fila de la tabla; complemento de <code>ultima-fila</code> .

Ejemplo: calcular la duración de cada franja horaria como la diferencia entre su hora de fin y la hora de fin de la fila anterior.

```
1 (col "Duracion" :computed
2   (_if (it-is-the-first-row)
3     (str "")
4     (subtract (col "Hora fin")
5               (previous-of (col "Hora fin")))))
```

Aritmética y texto

Forma	Para qué sirve
(add a b) / (subtract a b)	Suma y resta sobre valores numéricos o de celda.
(multiply a b) / (divide a b)	Multiplicación y división.
(time-add hora minutos)	Suma una cantidad de minutos a una hora Excel (<code>hora + minutos/1440</code>).
(concat a b)	Concatenación de texto: combina el contenido de dos expresiones en una sola cadena.

Ejemplo: calcular la hora de fin a partir de la hora de inicio y una duración fija en minutos.

```
1 (col "Hora fin" :computed
2   (time-add (col "Hora inicio") (col "Duracion min")))
```

Referencias avanzadas entre tablas

Forma	Para qué sirve
(tcol tabla nombre)	Como col, pero especifica explícitamente de qué tabla proviene la columna. Necesario cuando dos tablas de la misma hoja comparten un nombre de columna y hay que desambiguar.
(range desde hasta)	Rango de columnas de la tabla <i>actual</i> , sin necesidad de nombrarla. Equivalente de <code>trange</code> para la tabla en curso.
(lookup clave campo)	Búsqueda estilo VLOOKUP: dado el valor de clave en la fila actual, devuelve el valor de campo en la tabla de referencia.

Plantillas paramétricas

Cuando una misma estructura de tabla debe instanciarse varias veces con un detalle variable —por ejemplo, la misma tabla de franjas para cada día de la semana— se declaran *inst-params* en `def-table`:

```
1 (def-table franja-dia
2   ((turno "") (salon "") (grupo ""))
3   :inst-params (dia)
4   :computed
5   ((salon (tcol sala-disponible salon))))
```

Dentro de la definición, `(param nombre)` accede al valor que se pase al instanciar. Al construir la hoja se indica el parámetro como argumento con nombre:

```
1 (hoja "Semana"
2   (tabla franja-dia :dia 'lunes :data *datos-lunes*)
3   (tabla franja-dia :dia 'martes :data *datos-martes*))
```

Con este mecanismo el DSL genera cinco hojas con estructura idéntica y fórmulas parametrizadas con el nombre del día, a partir de una sola declaración `def-table`.

Detección de solapamiento dentro de la misma tabla

El tutorial usa `(exists (r :rows-of tabla) ...)` para buscar conflictos en una tabla *distinta*. La variante `:all-rows` detecta solapamientos dentro de la *propia* tabla:

```
1 (conditional-rendering
2   (exists (r :all-rows)
3     :matching (:col-a "Inicio" :col-b "Fin")
4     :any-overlap t)
5   :format '(:bg "#FF9999"))
```

Útil, por ejemplo, para marcar franjas que coinciden en la misma tabla de turnos de un único recurso.

Celdas de varias filas o columnas

Las opciones `:cell-height N` y `:cell-width N` de `def-table` hacen que cada fila lógica ocupe N filas (o columnas) físicas en el Excel generado. Es adecuado para horarios donde cada franja horaria debe abarcar varias celdas de la cuadrícula, o para encabezados que fusionan columnas.

```
1 (def-table horario-visual
2   ((hora "") (actividad "") (sala ""))
3   :cell-height 2) ; cada franja ocupa 2 filas físicas
```

Hojas con tablas apiladas verticalmente

La macro `hoja-v` es la versión vertical de `hoja`: apila las tablas una encima de otra en lugar de colocarlas en columnas. Se usa cuando las tablas tienen muchas columnas y el libro se orienta para impresión en apaisado, o cuando las tablas representan secciones consecutivas de un mismo listado.

```
1 (hoja-v "Resumen"
2   (tabla tabla-manana :data *datos-manana*)
3   (tabla tabla-tarde :data *datos-tarde*))
```

Referencias cruzadas entre hojas

Para libros con varias hojas que necesitan compartir datos, el DSL ofrece dos formas:

Forma	Para qué sirve
<code>(sheet-ref hoja plantilla-celda)</code>	Genera una referencia a una celda de otra hoja, expresada como plantilla de texto en la que <code>\${row-num}</code> se sustituye por el número de fila real.
<code>(cross-cell :sheet var :col col :row fila)</code>	Referencia a una celda concreta de otra hoja, identificada por su variable de hoja, columna y fila. Se usa cuando es necesario leer un campo de cada hoja en un bucle de generación.